2520030117
skill-09

EXPERIMENT – 9

Title:

Directory Navigation and Built-in Command Handling in Custom Shell

Aim:

To change directories, validate paths, update the working directory, handle errors, and maintain the previous directory.

Objective:

- To change directories.
- To validate directory paths.
- To update the working directory.
- To handle directory errors.
- To maintain the previous directory.
- To identify built-in commands.
- To execute commands inside the shell.
- To handle invalid commands.
- To maintain shell state.

Algorithm:

1. Start the custom shell.
2. Read the command from the user.
3. Check if the command is `cd`.
4. Validate the given directory path.
5. Change to the required directory.
6. Update the current working directory.
7. Handle invalid paths.
8. Maintain the previous directory.
9. Execute built-in commands directly.
10. Handle invalid commands.
11. Repeat until `exit` is entered.

Sample Commands:

pwd
cd ..
pwd

```
cd ~
pwd
cd /tmp
pwd
cd /wrong
```

Sample Output:

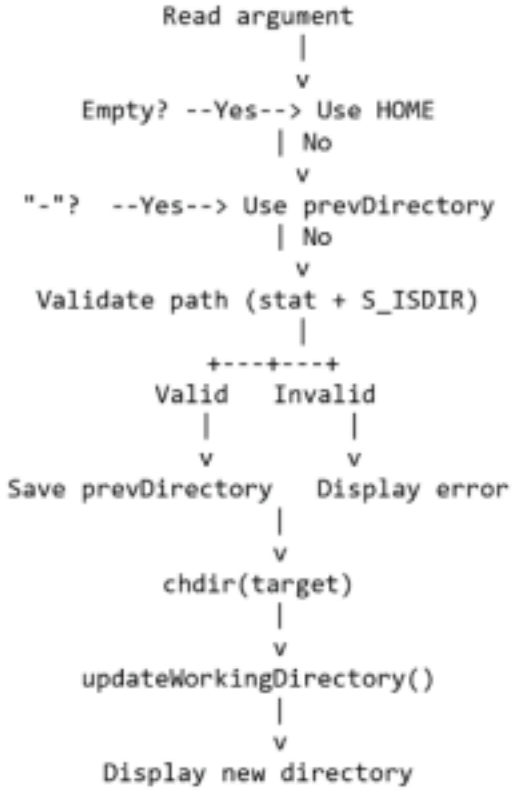
```
/home/vaishali
/home
/home/vaishali
/tmp
cd: No such file or directory
```

Result:

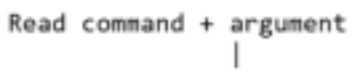
The custom shell successfully performs **directory navigation, path validation, working-directory updates, error handling, and built-in command execution**.

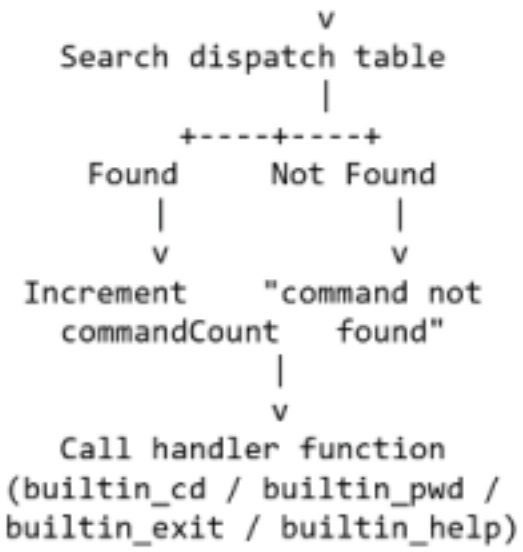
Design / Flow

Directory Navigation Flow



Dispatch Logic Flow





Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>

#define MAX 200

char prevDirectory[MAX] = "";
char currDirectory[MAX] = "";

/* ---------- Part 1: Directory Navigation ---------- */

int validatePath(const char *path)
{
    struct stat st;

    if (stat(path, &st) != 0)
    {
        printf("cd: %s: No such file or directory\n", path);
        return 0;
    }
    if (!S_ISDIR(st.st_mode))
    {
        printf("cd: %s: Not a directory\n", path);
        return 0;
    }
```

```
    return 1;
}

void updateWorkingDirectory()
{
    if (getcwd(currDirectory, sizeof(currDirectory)) == NULL)
        perror("getcwd");
}

void changeDirectory(const char *arg)
{
    char target[MAX];

    if (arg == NULL || strlen(arg) == 0)
    {
        char *home = getenv("HOME");

        if (home == NULL)
        {
            printf("cd: HOME not set\n");
            return;
        }

        strncpy(target, home, MAX);
    }
    else if (strcmp(arg, "-") == 0)
    {
        if (strlen(prevDirectory) == 0)
        {
            printf("cd: OLDPWD not set\n");
            return;
        }

        strncpy(target, prevDirectory, MAX);
        printf("%s\n", target);
    }
    else
    {
        strncpy(target, arg, MAX);
    }

    if (!validatePath(target))
        return;

    strncpy(prevDirectory, currDirectory, MAX);

    if (chdir(target) != 0)
```

```
    {
        perror("cd");
        return;
    }

    updateWorkingDirectory();
    printf("Directory changed to: %s\n", currDirectory);
}

void testNavigation()
{
    printf("\n--- Testing Directory Navigation ---\n");

    updateWorkingDirectory();

    printf("Starting directory: %s\n", currDirectory);

    changeDirectory("/tmp");
    changeDirectory("-");
    changeDirectory("/nonexistent_dir_xyz");
}

/* ---------- Part 2: Built-in Command Dispatcher ---------- */

int commandCount = 0;

void builtin_cd(char *arg)
{
    changeDirectory(arg);
}

void builtin_pwd(char *arg)
{
    (void)arg;
    updateWorkingDirectory();

    printf("%s\n", currDirectory);
}

void builtin_exit(char *arg)
{
    (void)arg;

    printf("Exiting shell...\n");

    exit(0);
}
```

```
void builtin_help(char *arg)
{
    (void)arg;

    printf("Built-in commands: cd, pwd, exit, help\n");
}

typedef struct
{
    const char *name;
    void (*func)(char *);
} DispatchEntry;

DispatchEntry dispatchTable[] = {
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"exit", builtin_exit},
    {"help", builtin_help}
};

int dispatchTableSize =
    sizeof(dispatchTable) / sizeof(DispatchEntry);

int isBuiltin(const char *cmd)
{
    for (int i = 0; i < dispatchTableSize; i++)
    {
        if (strcmp(cmd, dispatchTable[i].name) == 0)
            return 1;
    }

    return 0;
}

void executeBuiltin(char *cmd, char *arg)
{
    commandCount++;

    for (int i = 0; i < dispatchTableSize; i++)
    {
        if (strcmp(cmd, dispatchTable[i].name) == 0)
        {
            dispatchTable[i].func(arg);
            return;
        }
    }
```

```
    printf("%s: command not found\n", cmd);
}

void testDispatchLogic()
{
    printf("\n--- Testing Dispatch Logic ---\n");

    executeBuiltin("pwd", NULL);
    executeBuiltin("cd", "/tmp");
    executeBuiltin("help", NULL);
    executeBuiltin("foo", NULL);

    printf("Total commands dispatched: %d\n", commandCount);
}

/* ---------- Main ---------- */

int main()
{
    char line[MAX], cmd[MAX], arg[MAX];

    updateWorkingDirectory();

    printf("=== Part 1: Directory Navigation ===\n");

    testNavigation();

    printf("\n=== Part 2: Built-in Command Dispatcher ===\n");

    testDispatchLogic();

    printf("\n=== Interactive Mode (type 'exit' to quit) ===\n");

    while (1)
    {
        updateWorkingDirectory();

        printf("myshell:%s$ ", currDirectory);

        if (fgets(line, sizeof(line), stdin) == NULL)
            break;

        line[strcspn(line, "\n")] = '\0';

        if (strlen(line) == 0)
            continue;
```

```
        arg[0] = '\0';

        sscanf(line, "%s %[^\n]", cmd, arg);

        if (isBuiltin(cmd))
            executeBuiltin(cmd, arg[0] ? arg : NULL);
        else
            printf("%s: command not found (external execution not implemented)\n",
                   cmd);
    }

    return 0;
}
```